Documentación prueba técnica desarrolladora fullstack SIATA-EAFIT

Andrea Sanchez Castrillón.

JUSTIFICACIÓN DE LAS TECNOLOGÍAS EMPLEADAS Y PATRONES DE DISEÑO

1. Justificación de las tecnologías del backend

El backend de la solución se construyó con FastAPI como framework principal. Se eligió FastAPI porque permite desarrollar APIs REST de forma rápida, con documentación automática (OpenAPI/Swagger).

Para servir la aplicación se utiliza Uvicorn, un servidor ligero, recomendado por la propia documentación de FastAPI y adecuado tanto para desarrollo como para entornos de producción cuando se despliega detrás de un proxy o en un servicio en la nube.

La persistencia de datos se maneja con SQLAlchemy que permite definir los modelos de negocio en Python y abstraer las consultas a la base de datos. Se eligió PostgreSQL como motor de base de datos por su robustez, y soporte avanzado de tipos de datos necesarias para los modelos.

Las migraciones del esquema de base de datos se gestionan con Alembic, que trabaja de forma integrada con SQLAlchemy.

La validación de las peticiones y respuestas de la API se realiza con Pydantic. Los esquemas Pydantic garantizan que los datos que llegan a los endpoints cumplan tipos y reglas de negocio. Además, Pydantic se integra de forma nativa con FastAPI para la documentación.

La autenticación se implementa con JWT (JSON Web Tokens) usando la librería python-jose y passlib con bcrypt para el hasheo de contraseñas. Esta combinación permite mantener sesiones sin estado en el servidor: el frontend envía el token en cada petición y el backend lo valida sin necesidad de guardar sesiones en memoria o en base de datos. El uso de bcrypt para las contraseñas sigue las buenas prácticas de seguridad al tratarse de un algoritmo adaptativo y ampliamente recomendado.

La configuración sensible (cadena de conexión a la base de datos, clave secreta para JWT, etc.) se externaliza mediante variables de entorno y el uso de python-dotenv en desarrollo, lo que evita hardcodear credenciales en el código y facilita distintos entornos (desarrollo, staging, producción).

2. Justificación de las tecnologías del frontend

El frontend se desarrolló con React y TypeScript. React ofrece un modelo de componentes reutilizables, lo que permite estructurar la interfaz de usuario de forma clara y mantener el estado de la aplicación de manera predecible. TypeScript añade tipado estático, lo que reduce errores en tiempo de desarrollo y mejora la mantenibilidad del código cuando el proyecto crece.

Como herramienta de construcción se utiliza Vite que proporciona un servidor de desarrollo muy rápido gracias al uso de módulos nativos del navegador. Sustituye configuraciones más pesadas y se integra bien con React y TypeScript.

Para la interfaz de usuario se eligió Material-UI (MUI) que ofrece un conjunto de componentes listos para usar (tablas, formularios, diálogos, botones, etc.), lo que acelera el desarrollo y mantiene una apariencia profesional sin tener que implementar desde cero estilos y comportamientos de accesibilidad.

Las peticiones HTTP al backend se realizan con Axios. Axios simplifica el envío de peticiones REST, el manejo de cabeceras (por ejemplo, el token de autorización) y la gestión de errores de red, en comparación con la API nativa fetch.

El enrutado de la aplicación (páginas de login, registro, clientes, productos, bodegas, puertos, envíos) se gestiona con React Router. React Router permite definir rutas declarativas y proteger aquellas que requieren autenticación, separando claramente las vistas públicas de las privadas.

3. Patrones de diseño empleados

En el backend se sigue una arquitectura en capas. La capa de controladores (o rutas) recibe las peticiones HTTP y delega la lógica de negocio en una capa de servicios. Los servicios acceden a la base de datos a través de los modelos de SQLAlchemy y devuelven datos ya validados o transformados. Los esquemas Pydantic definen el contrato de entrada y salida de la API. Esta separación permite probar la lógica de negocio de forma aislada, reutilizar servicios desde distintos endpoints y modificar la persistencia o la presentación sin afectar al resto.

El patrón repositorio no se implementa de forma explícita como capa adicional; en su lugar, los servicios utilizan directamente la sesión de SQLAlchemy y los modelos para las operaciones CRUD. Esto mantiene el código simple y evita una capa de abstracción innecesaria dado el tamaño del proyecto.

En el frontend, los formularios de creación y ediciones (clientes, productos, bodegas, puertos, envíos) comparten una estructura común: una tabla de datos con acciones de editar y eliminar, un diálogo modal para crear o editar registros y un cuadro de confirmación para eliminar. Esta estructura se basó en componentes reutilizables: un componente de encabezado de página (título, subtítulo, botón de creación y mensaje de error) y un componente de tabla CRUD parametrizable por columnas, datos y mensajes. Con ello se evita duplicar código entre páginas y se facilita un comportamiento y apariencia uniformes.

La validación se realiza en dos niveles: en el frontend, para dar feedback inmediato al usuario (campos obligatorios, formatos de email, longitud de contraseña, reglas de negocio como el número de guía o la placa), y en el backend mediante los esquemas Pydantic y comprobaciones en la capa de servicios. Esta doble validación mejora la experiencia de usuario y garantiza que la API no acepte datos inválidos aunque se llame desde otro cliente, además de que permite evitar peticiones incorrectas al detener el envio de estas desde el front.

El manejo de errores se centraliza en el backend se muestran en códigos HTTP y mensajes coherentes (por ejemplo, 401 Unauthorized), que el frontend muestra al usuario mediante alertas. En el frontend, una función de utilidad (parseApiError) unifica la extracción del mensaje de error desde las respuestas de la API para mostrarlo de forma consistente en todas las páginas.

DEFINICIÓN DE BUENAS PRÁCTICAS

1. Seguridad y manejo de credenciales

Las credenciales y datos sensibles (cadena de conexión a la base de datos, clave secreta para firmar los tokens JWT, contraseñas de usuarios) no se almacenan nunca en el código fuente. Se utilizan variables de entorno. En desarrollo se emplea un archivo .env que no se sube al repositorio; en producción cada plataforma (Render, Vercel, Supabase) gestiona sus propias variables. De este modo se evita exponer secretos en Github y se facilita usar configuraciones distintas según el entorno.

Las contraseñas de los usuarios se almacenan en la base de datos solo después de hashearlas con bcrypt. Nunca se guardan en texto plano ni se envían por la API en las respuestas. El proceso de login compara el hash almacenado con el resultado de hashear la contraseña enviada por el cliente, sin posibilidad de recuperar la contraseña original desde la base de datos.

El acceso a los endpoints que requieren autenticación se controla mediante JWT. El token se envía en la cabecera Authorization y se valida en cada petición. Si el token es inválido o ha expirado, el servidor responde con código 401 y no se ejecuta la lógica protegida. Esta comprobación se centraliza en dependencias de FastAPI (por ejemplo, get_current_user), de modo que todas las rutas protegidas reutilizan la misma lógica y no se duplica el código de verificación.

2. Validación de datos

Toda la información que entra por la API se valida antes de usarla. En el backend, los esquemas Pydantic definen tipos, longitudes, formatos y reglas de negocio. Si los datos no cumplen las reglas, FastAPI responde automáticamente con 422 y un mensaje descriptivo, sin llegar a ejecutar la lógica de negocio.

En el frontend se aplica validación adicional en los formularios: campos requeridos, formatos (correo, número de guía, placa, número de flota) y reglas específicas (por ejemplo, precio mayor que cero, cliente y producto seleccionados en un envío). El usuario recibe feedback inmediato mediante mensajes de error junto a los campos, y el botón de envío puede deshabilitarse mientras haya errores. Esta doble validación (frontend y backend) mejora la experiencia de usuario y garantiza que la API siga siendo segura aunque se llame desde otro cliente o se desactive la validación del navegador.

3. Estructura y organización del código

En el backend se mantiene una separación clara entre la capa que recibe las peticiones HTTP (controladores o rutas), la capa de lógica de negocio (servicios) y la capa de persistencia (modelos y sesión de base de datos).

La sesión de base de datos se gestiona con el patrón de dependencia inyección: FastAPI inyecta la sesión en cada endpoint que la necesita y se asegura de cerrarla al finalizar la petición, incluso si ocurre una excepción. Así se evitan fugas de conexiones y se mantiene un único punto de configuración del acceso a datos.

Los cambios en el esquema de la base de datos se versionan con migraciones de Alembic. Cada cambio (nueva tabla, nueva columna, etc.) queda registrado en un archivo de migración y se aplica con un comando único (alembic upgrade head). No se modifican las tablas a mano en producción; de este modo el estado de la base de datos es reproducible y se puede auditar qué cambios se aplicaron y cuándo.

4. Reutilización y mantenibilidad en el frontend

Los elementos comunes entre las páginas de gestión (clientes, productos, bodegas, puertos, envíos) se extrajeron en componentes reutilizables: un componente de encabezado de página (título, subtítulo, botón de alta y alerta de error) y un componente de tabla CRUD parametrizable (columnas, datos, estado de carga, acciones editar y eliminar). Cada página define solo sus columnas y su formulario de creación/edición, reduciendo la duplicación de código y asegurando un comportamiento y una apariencia coherentes en toda la aplicación.

Los estilos y la estructura visual se apoyan en un sistema de diseño (Material-UI), de modo que no se inventan componentes específicos para cada página. Los colores principales y el espaciado se configuran de forma centralizada cuando es posible, para facilitar futuros cambios de identidad visual.

6. CORS y despliegue

El backend está configurado con CORS para permitir peticiones desde el origen del frontend desplegado (por ejemplo, el dominio de Vercel). Solo se permiten los orígenes necesarios y los métodos y cabeceras que la aplicación utiliza, en lugar de abrir el API a cualquier origen. Esto reduce el riesgo de que terceros consuman la API desde dominios no autorizados.

ARTEFACTOS DE DESPLIEGUE DE LA SOLUCIÓN

Flujo de despliegue

1. Preparación de la base de datos (Supabase):

- Crear el proyecto en Supabase.
- Obtener los datos de conexión (host, puerto, nombre BD, usuario, contraseña).
- Construir `DATABASE_URL` y configurarlo en Render.
- Desde el backend (local o en Render), ejecutar las migraciones de Alembic:
  - `alembic upgrade head`.

2. Despliegue del backend en Render:

- Crear servicio web vinculado al repositorio del backend.
- Definir:

  - Comando de build.
  - Comando de start (`uvicorn main:app --host 0.0.0.0 --port 8000`).
- Configurar variables de entorno (`DATABASE_URL`, etc.).
- Esperar a que Render construya la imagen y exponga la URL pública de la API.

3. Despliegue del frontend en Vercel:

- Crear proyecto en Vercel vinculando el repositorio que contiene la carpeta `frontend`.
- Configurar el comando de build (`npm run build`) y el directorio de salida (`dist`).
- Definir la variable `VITE_API_BASE_URL` apuntando a la URL pública del backend en Render.
- Ejecutar el despliegue; Vercel generará una URL pública para la aplicación.